Universidad de Oviedo

Escuela de Ingeniería Informática
Universidad de Oviedo

UNIVERSIDAD DE OVIEDO
ESCUELA DE INGENIERÍA INFORMÁTICA
GRADO EN INGENIERÍA INFORMÁTICA DEL SOFTWARE

TRABAJO DE FIN DE GRADO

IR-Board

Requirements Management Platform

Author:
Javier Carrasco Arango
Tutors:
Jorge Álvarez Fidalgo
Benjamín López Pérez

2026-06-16

Declaration of originality

I, , with DNI and UO294532, hereby declare that this work is completely original and all sources used during the development of it have been correctly cited. In Avilés, Asturias, on the xx of xx of 2026,

Signed:

Abstract

IR-Board is a Requirements Management Platform (RMP) designed to support the complete lifecycle of software requirements engineering. The project addresses the need for a unified environment that combines the rigor of traditional requirements specification standards with the flexibility of modern Agile methodologies. The platform is developed in accordance with the IEEE 830 and ISO/IEC/IEEE 29148 standards, providing structured support for the elicitation, documentation, validation, and maintenance of software requirements.

The system incorporates a hybrid documentation approach, enabling the management of traditional requirements engineering artifacts such as use cases, flowcharts, and decision tables, alongside Agile practices including user stories and story mapping. To ensure secure and granular access control, IR-Board implements a Relation-Based Access Control (ReBAC) model within a Zero-Trust architecture, allowing permissions to be determined dynamically according to user relationships with projects and functionalities.

In addition, the platform provides quality of life automations through controlled state transitions and approval workflows, improving traceability and governance. Collaborative features, including stakeholder management and concurrency control mechanisms, facilitate coordinated work among multiple users while preventing data conflicts. Finally, an integrated observability framework provides basic monitoring, operational metrics, and audit logging to support system reliability and administration. Through the integration of these capabilities, IR-Board offers a comprehensive solution for modern requirements engineering and project governance.

Keywords

Requirements Management Platform, Software Requirements Engineering, IEEE 830, ISO/IEC/IEEE 29148, Agile Methodology, Hybrid Documentation, User Stories, Use Cases, Relation-Based Access Control (ReBAC), Zero-Trust Architecture, Requirements Lifecycle Management, Traceability, Collaborative Engineering, Concurrency Control, Observability, Audit Logging, Project Governance, Workflow Automation.

About the document

This document follows a template provided by Jorge Álvarez Fidalgo.

It has been adapted and modified to fit the specific needs of the project during its development.

Special thanks to

Index

Figure Index

Table Index

Chapter 1. Introduction

1.1 Object

The primary object of this project is the design and implementation of **IR-Board**, a comprehensive Requirements Management Platform (RMP) designed to support the full lifecycle of software requirements engineering. The system aims to bridge the gap between traditional documentation standards and modern agile methodologies, providing a centralized environment for eliciting, refining, and managing requirements.

Key objectives of the system include:

- **Methodological Compliance:** Implementing a framework that follows international standards for requirements specification, specifically the **IEEE 830** guidelines and the **ISO/IEC/IEEE 29148** standard.
- **Hybrid Documentation Support:** Providing tools for both traditional modeling (use cases, flowcharts, and decision tables) and Agile practices (user story mapping and management).
- **Relation-Based Access Control (ReBAC):** Developing a sophisticated security model where permissions are not merely role-based but determined by the dynamic relationship between users and specific entities (Projects and Functionalities), implemented through a **Zero-Trust** architecture.
- **Lifecycle and State Management:** Automating the management of requirement states (Pending Approval, Approved, Deactivated, etc.) and project lifecycles to ensure data integrity and traceability.
- **Collaborative Engineering:** Facilitating stakeholder management and real-time concurrency control to prevent data conflicts during collaborative editing sessions.
- **System Observability:** Integrating a high-standard monitoring stack to provide administrators with full visibility into the infrastructure's health and security audit logs.

1.2 Background

Software requirements engineering is a fundamental phase in the software development lifecycle, as it defines what a system must achieve before design and implementation begin. It also helps bridge the gap between client needs and the technical rigor required to reduce misunderstandings, contractual ambiguity, and potential legal issues. To ensure quality and consistency, standardized approaches such as IEEE 830 and ISO/IEC/IEEE 29148 have been widely adopted. These standards emphasize structured documentation, traceability, validation, and completeness of requirements, making them particularly suitable for large-scale and regulated software systems.

In contrast, modern software development has increasingly shifted toward Agile methodologies, which prioritize iterative development, rapid feedback, and lightweight documentation. Techniques such as user stories and story mapping allow teams to respond quickly to changing requirements and stakeholder needs. How-

ever, Agile approaches often reduce formal structure, which can make long-term traceability and governance more challenging.

As a result, many organizations now operate in hybrid environments where both traditional and Agile requirements engineering practices are used simultaneously.

1.3 Current state

Despite the coexistence of traditional and Agile methodologies, existing requirements management tools tend to favor one approach over the other. Traditional tools focus heavily on structured documentation and compliance with formal standards, while Agile-oriented platforms emphasize backlog management and iterative workflows. This separation often leads to fragmented processes, requiring teams to use multiple disconnected tools.

This fragmentation can result in reduced traceability, inconsistencies between requirement representations, and difficulties in maintaining lifecycle integrity across different development methodologies. Additionally, many existing systems provide limited support for advanced access control models, relying mainly on role-based access control (RBAC), which lacks flexibility in dynamic, collaborative environments.

At the same time, there is a broad ecosystem of commercial requirements management and product lifecycle tools that aim to address these challenges by offering more integrated workflows. Platforms such as IBM Engineering Requirements Management DOORS Next, Jama Connect, Polarion ALM, and modern lifecycle tools like Atlassian Jira (with extensions) or Azure DevOps provide end-to-end support for requirements tracking, traceability, collaboration, and integration with development pipelines. These systems are often highly mature and feature-rich, supporting compliance workflows, test management, and cross-team collaboration within large enterprises.

However, despite their extensive capabilities, these solutions are typically proprietary, complex to configure, and associated with significant licensing costs. They are often optimized for enterprise-scale environments, which makes them less accessible for small teams, academic projects, or organizations seeking flexible customization. Furthermore, while they provide strong workflow integration, their support for modern architectural security models such as Relation-Based Access Control (ReBAC) and Zero-Trust principles remains limited or heavily abstracted.

At the same time, there is an increasing demand for more sophisticated system capabilities such as real-time collaboration, automated workflow and state management, and integrated observability for monitoring system health and security. Emerging security models such as ReBAC and Zero-Trust architectures are not yet widely adopted in most requirements management platforms.

Within this landscape, open-source alternatives remain relatively limited in scope and maturity. Existing open-source tools tend to focus on either issue tracking or lightweight Agile backlog management rather than providing a full requirements engineering lifecycle aligned with formal standards such as IEEE 830 and ISO/IEC/IEEE 29148. As a result, there is a clear gap in the market for an open, exten-

sible, and standards-compliant platform that combines structured requirements engineering, Agile methodologies, advanced access control mechanisms, and collaborative lifecycle management within a single coherent system.

1.4 Definitions and Abbreviations

1.4.1 General concepts

- **RMP (Requirements Management Platform)**: A software system designed to support the full lifecycle of software requirements, including elicitation, specification, validation, traceability, and change management.
- **Requirements Engineering (RE)**: A discipline of software engineering focused on defining, documenting, and maintaining software requirements throughout the system lifecycle.
- **IEEE 830**: A legacy IEEE standard for software requirements specification, defining structure and content of Software Requirements Specifications (SRS).
- **ISO/IEC/IEEE 29148**: The current international standard for requirements engineering processes and requirements specification quality.
- **Agile Methodology**: An iterative software development approach that emphasizes incremental delivery, collaboration, and adaptability to change.
- **User Story**: A short description of a feature from an end-user perspective, commonly used in Agile development.
- **Use Case**: A structured description of a system's behavior in response to external actors.

1.4.2 Security and access control

- **ReBAC (Relation-Based Access Control)**: An authorization model where access permissions are determined by relationships between entities (e.g., user–project, user–role–resource), rather than static roles.
- **RBAC (Role-Based Access Control)**: A traditional access control model where permissions are assigned to roles, and users inherit permissions through assigned roles.
- **Zero-Trust Architecture**: A security model that assumes no implicit trust in any user or system component, requiring continuous verification of identity and permissions.
- **OIDC (OpenID Connect) (implied via Ory ecosystem context)**: An authentication layer built on OAuth 2.0 used for identity verification.

1.4.3 Protocols and data modelling concepts

- **SMTP (Simple Mail Transfer Protocol)**: The standard protocol used for sending emails between servers.
- **Internal Unique Identifier (ID)**: A system-generated identifier used internally to uniquely identify an entity in the database.
- **Dynamic Identifier**: A human-readable structured identifier generated from relationships and context (e.g., FR-UM-001).
- **Slug**: A URL-safe unique string identifier, typically used for routing or external references.
- **Floating Point Ordering**: A technique for ordering entities (e.g., requirements) using floating-point values to allow efficient reordering without full renumbering.

- **Entity**: A domain object within the system such as a project, requirement, stakeholder, or document.
- **Audit Log**: A record of system actions used for traceability and accountability.

1.4.4 Abbreviations

- API – Application Programming Interface
- DNS – Domain Name System
- UI – User Interface
- UX – User Experience
- SRS – Software Requirements - Specification
- MTA – Mail Transfer Agent
- TLS – Transport Layer Security
- JWT – JSON Web Token
- CLI – Command Line Interface
- CI/CD – Continuous Integration / Continuous Deployment

1.5 Scope

The scope of this project is the design and implementation of IR-Board, a Requirements Management Platform focused on supporting the complete lifecycle of software requirements engineering. The system aims to provide a centralized environment where software projects can define, organize, validate, and maintain requirements while preserving traceability between the different elements involved in the requirements process.

The project covers the analysis, design, development, and validation of the core platform functionalities required to manage software requirements in a structured manner.

The implemented solution combines traditional requirements engineering practices with selected Agile-oriented concepts, allowing users to manage requirements, stakeholders, documents, and related project information from a unified interface.Furthermore, it includes support for the complete management lifecycle of project entities. This includes the creation, modification, validation, deactivation, and archival of projects, functionalities, requirements, stakeholders, and associated documentation. The platform provides mechanisms to maintain the relationships between these entities, enabling bidirectional traceability and ensuring that changes affecting one element can be identified across related elements.

About document management, the system allows users to upload, associate, update, and manage documents linked to project entities. These documents contribute to requirements traceability by allowing requirements and other elements to reference supporting material throughout the development lifecycle.

Another objective within the scope of the project is the implementation of a secure architecture following Zero-Trust principles. The system incorporates identity management, authentication, and authorization mechanisms designed to prevent implicit trust between components. Fine-grained access control is implemented through a Relation-Based Access Control (ReBAC) approach, allowing permissions to be determined dynamically according to relationships between users, projects,

functionalities, and other system entities. More on the selection of a ReBAC access control on the section alternatives analysis. About the user management, the system covers user invitation, authentication workflows, credential management, and permission assignment. Users can participate in projects according to their assigned relationships and responsibilities, enabling collaborative work while maintaining controlled access to each project.

The project also includes the implementation of basic system observability capabilities. The platform provides mechanisms for collecting operational information such as application logs and relevant system metrics. These capabilities are intended to assist development, debugging, maintenance, and basic operational monitoring of the deployed solution.

The platform is designed with extensibility and future integration in mind. Although some external services are not implemented as part of this project, the architecture has been prepared to allow their incorporation without requiring significant changes to the core system. This includes the possibility of replacing development components with production-ready alternatives when deploying the system in a real environment.

The following elements are considered outside the scope of this project:

- Full production-grade security hardening of the observability infrastructure is not included. Although logging and metric collection mechanisms are implemented, securing, isolating, auditing, and managing the observability ecosystem itself is considered an infrastructure operation outside the objectives of this project.
- Integration of the observability stack as part of the business application is not included. The platform exposes the necessary information for monitoring, but advanced dashboards, alerting systems, automated incident response, and operational intelligence, included in comprehensive and unified panels within the frontend are outside the project scope.
- Integration with external third-party platforms is not included. This includes external project management tools, development platforms, issue trackers, or other enterprise systems.
- Production email infrastructure is not included. During development, email-related functionality is supported through development-oriented components intended for testing purposes, that is, a local development test server called mailpit. Configuration and deployment with real SMTP providers, domain verification, email reputation management, SPF, DKIM, DMARC, and production mail delivery processes, all necessary to allow a mail to even reach the spam folder when self hosting, are outside the scope of this project.
- Deployment automation for large-scale production environments is not included. While the system is containerized and designed for reproducible deployment, advanced orchestration, scaling strategies, and cloud-specific infrastructure management are considered future work.

The final result of this project is therefore a functional and extensible prototype of a requirements management platform that demonstrates secure architecture

design, requirements lifecycle management, traceability, collaboration capabilities, and basic operational visibility, while leaving production infrastructure concerns and external ecosystem integrations as future extensions.

1.6 Assumptions and Constraints

The development of IR-Board is based on several assumptions and constraints regarding the environment, users, and expected usage of the platform.

It is assumed that the system will primarily be used by software development teams or academic/project environments where requirements need to be collected, structured, reviewed, and maintained throughout a software lifecycle. The platform is not intended to replace complete enterprise Application Lifecycle Management (ALM) suites, but rather provide a focused requirements engineering environment combining formal documentation practices with Agile techniques.

The project is constrained by the available development time and the scope of a bachelor's thesis. Therefore, some enterprise-level functionalities commonly found in commercial requirements management platforms are not implemented.

It is assumed that users interacting with the system have basic knowledge of software development concepts and requirements engineering terminology. Although the platform provides mechanisms to guide the creation and validation of requirements, it does not attempt to automatically determine whether a requirement is correct from a business perspective; this responsibility remains with stakeholders and project members, and therefore those actions are fundamentally manual by design.

The system assumes that requirements are created and maintained following a structured lifecycle. Requirements may evolve over time, and changes are expected to occur through controlled modification, validation, and approval processes rather than direct deletion. Therefore, the platform assumes that maintaining historical information and traceability is more valuable than permanently removing data in a quick manner.

The project assumes that the group using the system may have different projects at a time, operated by different groups of developers, and therefore static global roles are unfit for access control.

The project assumes that external security services such as identity management, authentication, and authorization providers can be integrated through standardized interfaces. Therefore, solutions such as the Ory ecosystem are treated as replaceable infrastructure components rather than business logic dependencies.

It is assumed that PostgreSQL provides enough flexibility for the domain model, including structured relational data and semi-structured information through JSONB fields. The system does not require a dedicated NoSQL database because the main entities require strong consistency and traceability.

The document management subsystem assumes that uploaded documents act mainly as requirement-related artifacts. Advanced document processing, version

control systems, collaborative document editing, or automatic content extraction are outside the expected functionality.

The observability layer assumes that basic logging, metrics, and monitoring capabilities are sufficient for system administration and debugging. The project does not assume the need for security monitoring, automated incident response, or a complete observability platform.

The deployment environment assumes containerized execution. Components are expected to run through container orchestration tools such as Docker Compose during development and testing, allowing the same architecture to be adapted to more advanced deployment environments if required.

Chapter 2. Theoretical Background

To better understand the system developed, here is a brief summary of the concepts related and used on it.

2.1 Software requirements engineering

Requirements engineering is the structured and systematic approach to formalizing the needs and expectations of stakeholders for a system. It encompasses different processes for identifying, eliciting, analyzing, specificating, validating and their management.

2.1.1 Concepts

But first, it is necessary to explain what a **stakeholder** is. A stakeholder is someone with a "stake" or share on a project, or in general, someone interested in it. Stakeholders are the ones who, directly or indirectly define what a system must do for it to succeed. Common examples of stakeholders are end users, investors, industry competitors, and the development team. Understanding the stakeholders related to a project and their needs is imperative for a project to be accepted and provide value.

A **requirement** represents a formal, documented statement describing a capability, condition, or constraint that a system must satisfy. Requirements act as the communication bridge between stakeholders and the development team, allowing the expected behavior of a system to be understood, evaluated, and implemented.

2.1.2 Criteria

Generally, requirements are classified according to different criteria, including their abstraction level and whether they are functional or non-functional. Regarding abstraction, requirements are commonly organized in a hierarchical structure, where high-level requirements are progressively refined into more specific ones.This hierarchy allows requirements to be represented using identifiers, commonly composed of a unique identifier followed by a structured notation for their refinements. Sub-requirements can be nested within parent requirements, often using dot-separated numbering schemes that represent their position within the hierarchy.

At the highest level, **business requirements** describe the objectives and needs that motivate the development of a system. They are closer to the concerns of stakeholders and are therefore usually expressed in a more abstract way, making them easier for non-technical stakeholders to understand and validate. As requirements are refined into lower levels of abstraction, they become increasingly detailed and closer to the technical aspects of the system. These **technical requirements** describe specific behaviors, constraints, or implementation-related aspects necessary to satisfy the higher-level objectives.

This separation between abstraction levels facilitates communication between stakeholders and development teams, while also improving requirements traceability by allowing each detailed requirement to be linked back to the original business need that originated it.

Regarding the other criteria, **functional requirements** describe the behaviors and services that the system must provide, defining what the system should do. Examples include allowing a user to create an account, generating reports, or managing project information. **Non-functional requirements**, on the other hand, define quality attributes and constraints that affect how the system performs. These include aspects such as security, performance, reliability, usability, and maintainability.

2.1.3 Lifecycle

The requirements engineering process usually follows several interconnected activities:

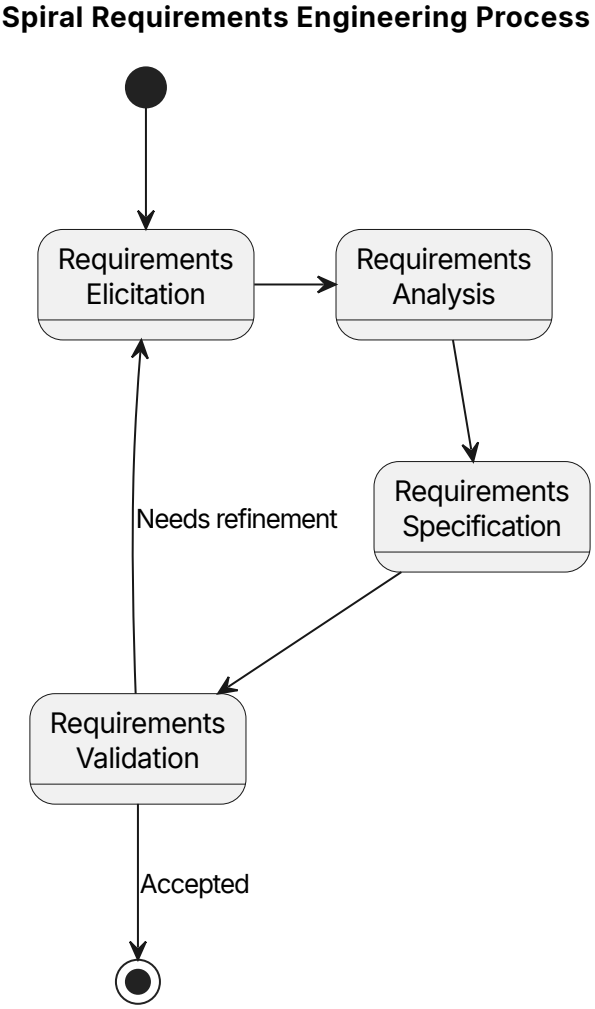

Figure 1: Requirement engineering processes

The first phase is **requirements elicitation**, where information is gathered from stakeholders through techniques such as interviews, observation, workshops, or analysis of existing documentation. The objective of this phase is to understand the problem domain, its scope, and identify the actual needs behind the requested system. Due to the inherently human-centered nature of requirements elicitation, many of these techniques are influenced by methods from fields such as social sciences, qualitative research, and human-computer interaction. This is because requirements are not always explicitly known or easily communicated by stake-

holders; they often need to be discovered through structured interaction, analysis, and interpretation.

After elicitation, requirements are analyzed and refined. During **requirements analysis**, ambiguous, conflicting, or incomplete requirements are identified and resolved. This stage also involves prioritizing requirements according to factors such as business value, feasibility, cost, and risk.

Once analyzed, requirements are documented during the **requirements specification** phase. The purpose of specification is to create a clear and structured representation of the requirements, reducing misunderstandings between stakeholders and developers. A well-written requirement should be understandable, consistent, testable, and traceable throughout the development process.

Requirements must also be validated before being considered complete. **Requirements validation** ensures that documented requirements accurately represent stakeholder needs and that they are realistic within the technical and organizational constraints of the project. Validation activities may include reviews, prototypes, and acceptance criteria definition. Logically, part of the requirements may be validated while others may need refinement. For this reason, the process is modelled as a spiral, iterating over the different phases until accepted.

Finally, requirements engineering includes **requirements management**, which handles changes to requirements during the lifecycle of a project. Software systems frequently evolve due to changes in business needs, regulations, technology, or user expectations. Managing these changes ensures that modifications are controlled and that the relationship between requirements, design decisions, implementation, and testing remains clear.

2.1.4 Traceability

A key concept within requirements management is **traceability**. Requirements traceability refers to the ability to follow the relationship between a requirement and other elements associated with it, such as stakeholders, design components, implementation artifacts, tests, or related requirements. Traceability improves impact analysis, change management, and verification by providing visibility into how modifications affect the overall system.

Forward traceability ensures that every requirement can be followed from its original stakeholder need towards its implementation and verification artifacts. This helps confirm that all requested functionality has been addressed.

Backward traceability performs the opposite analysis, allowing developers and analysts to determine why a specific implementation element exists and which requirement originated it. This is particularly useful for preventing unnecessary functionality and supporting maintenance activities.

Horizontal traceability refers to relationships between artifacts at the same abstraction level or between parallel elements of the system. Unlike hierarchical decomposition, horizontal traceability focuses on dependencies, interactions, and consistency between related elements. Examples include linking requirements that depend on each other, connecting a requirement with its related stakeholders, as-

sociating documents or models with the requirements they describe, or identifying conflicts and overlaps between different requirements. This form of traceability is especially important in complex systems, where a modification to one element may affect several other elements that are not directly above or below it in the hierarchy.

One common approach to implementing these concepts is **identifier-based** traceability, where each requirement is assigned a unique identifier that remains stable throughout its lifecycle. These identifiers provide a direct reference mechanism between requirements, design elements, source code components, test cases, documentation, and other related artifacts. In this project, unique and semantically meaningful identifiers are used. For functional requirements, the identifier is composed of the initials of the requirement category or containing functionality, followed by a dot-separated numbering scheme that represents its position within the requirement hierarchy. This structure allows requirements to be located and understood without inspecting their complete content.

Another commonly used strategy is **traceability matrices**, where relationships between different artifacts are represented in a structured table. For example, a requirements-to-test matrix can demonstrate that every requirement has associated validation activities, while a requirements-to-design matrix can show how each requirement is translated into system components.

2.2 Requirements Documentation Standards

Requirements documentation standards provide guidelines and recommendations for creating consistent, complete, and understandable descriptions of software requirements. Their main objective is to reduce ambiguity between stakeholders and development teams by defining common structures, terminology, and practices for representing system needs.

2.2.1 IEEE 830

The IEEE 830 standard, formally known as **IEEE Recommended Practice for Software Requirements Specifications**, defines recommendations for writing Software Requirements Specifications (SRS).

The standard establishes characteristics that a good requirement specification should have. A requirements document should be:

- **Correct:** The documented requirements should represent the actual needs of the stakeholders and system objectives.
- **Unambiguous:** Each requirement should have only one possible interpretation, avoiding unclear or subjective language.
- **Complete:** All relevant requirements, constraints, and system behaviors should be included.
- **Consistent:** Requirements should not contradict each other or define incompatible system behavior.
- **Verifiable:** Each requirement should be testable through objective validation methods.
- **Modifiable:** The document structure should allow changes without introducing inconsistencies.

- **Traceable:** Each requirement should be uniquely identifiable and linked to its origin and related artifacts.

Although modern Agile methodologies often favor lighter documentation approaches, the principles introduced by IEEE 830 remain applicable, especially regarding clarity, traceability, and verification.

2.2.2 ISO/IEC/IEEE 29148

The ISO/IEC/IEEE 29148 standard, **Systems and software engineering - Life cycle processes - Requirements engineering**, provides a more modern and comprehensive framework for requirements engineering. While IEEE 830 focuses mainly on the structure of the requirements specification document, ISO/IEC/IEEE 29148 covers the complete requirements lifecycle.

This standard defines processes for requirements definition, analysis, management, validation, and maintenance throughout the entire system lifecycle. It emphasizes that requirements should not be considered static documents, but rather managed entities that evolve together with the system.

ISO/IEC/IEEE 29148 introduces concepts such as:

- **Stakeholder requirements and system requirements:** The criteria about requirement abstraction based on levels seen on the criteria section. Stakeholder requirements are the equivalent of lower level, project-specific business requirements, as in some contexts, a business requirement may be broader, to affect the whole business.
- **Requirements management:** The continuous process of controlling requirement changes, relationships, versions, and status throughout the lifecycle.
- **Requirements verification and validation:** Activities that ensure requirements are correctly defined (verification) and that they represent the intended system (validation).

The standard also reinforces the importance of requirements attributes beyond their textual description. These attributes may include identifiers, priority, source, rationale, status, dependencies, and verification criteria. These elements support traceability and enable better impact analysis when requirements change.

Compared to IEEE 830, ISO/IEC/IEEE 29148 provides a broader lifecycle-oriented perspective, making it more suitable for modern development environments where requirements continuously evolve. For this reason, IR-Board follows the principles of both standards: using structured requirement documentation inspired by IEEE 830 while applying lifecycle management concepts defined by ISO/IEC/IEEE 29148.

2.3 Agile methodologies

Agile methodologies are software development approaches that prioritize adaptability, continuous feedback, and incremental delivery over rigid, sequential planning. They emerged as a response to traditional development models, where requirements were often defined completely at the beginning of the project and changes were expensive to introduce later.

Their approach is based on the idea that software requirements evolve as stakeholders gain a better understanding of their needs and as the project progresses. Instead of attempting to define the complete system beforehand, Agile teams work through short development iterations or sprints, frequently delivering functional increments of the system and incorporating feedback from users and stakeholders.

Common principles of Agile development include close collaboration between developers and stakeholders, continuous improvement, prioritization of valuable features, and responding to changes rather than strictly following an initial plan.

Although Agile methodologies generally favor lightweight documentation, they do not eliminate the need for requirements management. Instead, requirements are commonly represented through simpler and more flexible artifacts that can evolve during development, like user stories.

User stories are an Agile technique used to describe system requirements from the perspective of the user or stakeholder. Rather than focusing on technical implementation details, user stories express the expected value and purpose of a feature. They generally follow a structure like so:

As a [type of user], I want [goal], so that [reason or benefit].

2.4 Security principles

Software security principles are fundamental concepts used to design systems that protect data, users, and operations against unauthorized access, misuse, or failure. Security is not only implemented through specific mechanisms but also through architectural decisions and development practices.

A common principle is the principle of **least privilege**, which states that users and components should only have the permissions required to perform their intended tasks. This reduces the potential impact of compromised accounts or services.

Another important principle is defense in depth, where multiple independent security layers are applied instead of relying on a single protection mechanism. If one layer fails, additional controls can still prevent or limit damage.

Modern security approaches also emphasize concepts such as secure-by-design, where security considerations are incorporated from the beginning of the system lifecycle rather than added as a final step.

2.5 Software architectures

Software architecture describes the high-level structure of a software system, including its components, their responsibilities, and the communication mechanisms between them. Architectural decisions influence scalability, maintainability, security, and the ability of the system to evolve. It not only defines how software is organized internally, but also how different parts of the system interact with each other and with external services.

2.5.1 Monolithic Architecture

A monolithic architecture is a software architecture where the different system components are developed, deployed, and executed as a single application unit.

In a monolithic system, business logic, data access, authentication, and other application concerns usually exist within the same deployment artifact. This approach is simple to develop and deploy, especially for smaller systems, as communication between components occurs through internal method calls.

However, as the system grows, monolithic architectures can become harder to maintain because changes in one component may require redeploying the entire application. Scaling individual parts independently is also more difficult, since the complete application is scaled as a single unit.

2.5.2 Microservices Architecture

A microservices architecture divides an application into a collection of smaller, independent services. Each service is responsible for a specific business capability and communicates with other services through well-defined interfaces.

Microservices allow independent deployment, scaling, and development of different system components. They also provide stronger isolation, as failures in one service can potentially be contained without affecting the entire system.

However, this approach introduces additional complexity, including network communication, service discovery, distributed data management, monitoring requirements, and more complex deployment processes.

For this reason, microservices architectures are often combined with supporting infrastructure such as API gateways, centralized logging, authentication services, and monitoring systems.

2.6 Observability

Observability is the ability to understand the internal state and behavior of a software system by analyzing its external outputs. It allows operators and developers to detect failures, investigate problems, and understand system performance.

Modern distributed systems require observability because failures are often not limited to a single component. Instead, they may result from interactions between several services, infrastructure components, or external dependencies.

The three main pillars of observability are logs, metrics, and traces.

2.6.1 Logs

Logs are timestamped records of events produced by applications and infrastructure components. They provide detailed information about what happened at a specific point in time. Examples of logged events include application errors, authentication attempts, configuration changes, or important business operations.

Logs are especially useful for debugging and investigating incidents because they provide contextual information about individual events.

2.6.2 Metrics

Metrics are numerical measurements that represent the state or performance of a system over time. Unlike logs, which describe individual events, metrics provide aggregated information that can be analyzed through trends and comprehensive dashboards.

Common examples include CPU usage, memory consumption, request latency, error rates, or the number of active users.

2.6.3 Traces

Traces represent the path of a request as it travels through different components of a system. They are particularly important in distributed architectures where a single user action may involve multiple services.

A trace is composed of multiple spans, where each span represents an operation performed by a specific component. By analyzing traces, developers can identify bottlenecks, latency sources, or failures across service boundaries.

Together, logs, metrics, and traces provide a complete view of system behavior, supporting reliability, maintenance, and operational decision-making.

Chapter 3. Feasibility Study and Alternatives Analysis

Before starting the implementation of IR-Board, an analysis of the technical feasibility and the different alternatives available was performed. The objective of this analysis was to identify the most appropriate technologies and architectural decisions according to the characteristics of the system to be implemented, the expected complexity, the security requirements, and the available development resources.

The main factors considered during the evaluation were maintainability, scalability, security, development complexity, ecosystem maturity, integration capabilities, and suitability for the requirements management domain.

Given the collaborative nature of requirements engineering systems, the decision was made to develop IR-Board as a web-based platform rather than as a standalone desktop application. A web architecture provides easier deployment, multiplatform access, and a better foundation for collaborative workflows where several users may interact with the same project simultaneously.

The selected architecture separates the frontend and backend into independent projects. This separation improves maintainability and allows each layer to evolve independently. Additionally, it facilitates future deployment scenarios where the frontend could be served independently or replaced without requiring modifications to the core business logic.

3.1 Deferral or self-implementation of security environment

Security was identified as a critical aspect of the system due to the collaborative nature of requirements management and the need for controlled access between users, projects, and functionalities.

Two approaches were considered: implementing security mechanisms directly inside the application, or delegating security responsibilities to specialized external components.

Criteria	Custom implementation	External security ecosystem
Control	Maximum control	Controlled through configuration
Development effort	Very high	Lower
Security assurance	Depends on implementation quality	Based on mature solutions
Maintainability	Higher responsibility	Separated components

Table 1: Security implementation alternatives comparison

A custom implementation would provide maximum control over authentication, authorization, and session management. However, implementing a complete security environment would significantly increase the complexity of the project. Features such as secure password management, session handling, authorization policies,

token management, and protection against common vulnerabilities require extensive development and testing.

For this reason, the project decided to use specialized open-source identity and security components from the Ory ecosystem. Their specific licensing is documented over on the appendices.

Ory Kratos is responsible for identity management and user sessions. Ory Oathkeeper acts as an authorization gateway, validating requests before they reach internal services. Ory Keto provides relationship-based access control (ReBAC), which is particularly appropriate for the project because permissions depend on relationships between users, projects, and functionalities.

Similarly, Traefik was selected as the API gateway and entry point due to its maturity and previous experience with the technology.

The use of these components follows the security principle of defense in depth, separating responsibilities and reducing the amount of security-critical code maintained by the project.

3.2 Backend framework selection

The backend was approached as a system exposing a REST API from the beginning. Three main alternatives were considered: Spring Boot, Express.js, and FastAPI.

Framework	Advantages	Disadvantages
Spring Boot	Mature ecosystem, strong security support, dependency injection, Spring Data JPA integration, suitable for complex domain models	Higher initial complexity and more verbose configuration compared with lightweight alternatives
Express.js	Simple architecture, rapid development, large JavaScript ecosystem, direct compatibility with React and TypeScript	Minimal framework approach requires manual decisions for architecture, security, validation, and application structure
FastAPI	Modern API design, automatic OpenAPI generation, type-based validation, high performance, simple development model	Less opinionated architecture, fewer enterprise-oriented patterns, additional design decisions required for complex systems

Table 2: Backend framework alternatives comparison

3.2.1 Spring Boot

Spring Boot was selected as the backend framework due to its maturity, strong ecosystem, and extensive support for enterprise-oriented applications. Its opinionated structure provides several development and security advantages, including dependency management, validation support, integrated configuration management, security modules, and mature database access through Spring Data JPA.

Additionally, the IR-Board domain requires complex relationships between entities, lifecycle management, state transitions, and strict data consistency. The object-relational mapping capabilities provided by JPA allow these relationships to be represented naturally while maintaining control over the resulting database structure. The Spring ecosystem also provides mature integration possibilities with authentication systems, authorization layers, and observability tools.

3.2.2 Express.js

Express.js was considered due to its simplicity, lightweight nature, and strong integration with JavaScript-based frontend ecosystems. Since the frontend was planned using React and TypeScript, using a JavaScript-based backend would allow sharing the same programming language across the stack, potentially simplifying development and reducing context switching.

However, Express.js is intentionally minimal and provides fewer architectural constraints. This means that important aspects such as project structure, validation, security practices, dependency management, and error handling need to be manually defined. For a system where security, traceability, and long-term maintainability are important objectives, this additional responsibility increases implementation complexity and risk.

3.2.3 FastAPI

FastAPI was also considered as a modern alternative for REST API development. Its use of Python type hints and Pydantic models provides automatic validation, clear API contracts, and automatic OpenAPI documentation generation. Additionally, its asynchronous architecture can provide excellent performance for I/O-heavy applications while maintaining a relatively simple development experience.

Despite these advantages, FastAPI was not selected. Although it is a strong framework for API-focused applications, it provides fewer built-in architectural conventions for large domain-oriented systems compared with Spring Boot. More decisions regarding dependency injection patterns, application organization, security integration, and persistence architecture would need to be defined independently. While this flexibility is valuable in many contexts, the additional design decisions would increase the complexity of maintaining a consistent architecture.

The final decision was Spring Boot because the additional structure, mature security ecosystem, database integration capabilities, and established enterprise patterns better match the requirements of IR-Board. The framework reduces implementation risks and development efforts while providing a solid foundation for future expansion.

3.3 Frontend framework selection

The frontend is responsible for presenting the requirements management interface and enabling user interaction with complex entities such as requirements, documents, stakeholders, and project structures. Due to the collaborative and data-intensive nature of the system, the frontend needed to support reusable components, responsive layouts, maintainable state management, and a flexible design system.

The main frontend framework alternatives considered were React and Angular.

React with TypeScript was selected as the frontend framework due to its flexibility, ecosystem maturity, and previous experience with the technology. React provides a component-based development model, allowing the creation of reusable interface elements and facilitating the development of complex interactive views. The use of TypeScript adds static typing, reducing potential errors and improving maintainability in a project with a large number of entities and interactions.

Angular was considered because it provides a complete frontend framework including routing, dependency injection, and a predefined application structure. This approach improves consistency and can simplify development in large teams by reducing the number of architectural decisions required during implementation. However, Angular's broader framework scope, stricter conventions, and larger set of integrated concepts introduce additional complexity compared with lighter approaches. Since IR-Board requires flexibility in interface design and integration with external services, React offered a better balance between structure, adaptability, and control over the selected tooling.

Criteria	React + TypeScript	Angular
Architecture	Flexible component-based approach; additional tooling chosen according to project needs	Complete framework with predefined application structure
Flexibility	High; developers choose supporting libraries and patterns	Moderate; framework conventions guide implementation
Learning curve	Moderate; requires selecting additional tools	Higher initially due to wider framework scope
Ecosystem	Large ecosystem and extensive third-party integrations	Large ecosystem with official tooling
Decision	Selected	Rejected

Table 3: Frontend framework alternatives comparison

3.4 Database selection

The system requires persistent storage for projects, requirements, users, documents, relationships, and lifecycle information. The database solution needed to provide reliable persistence, transactional guarantees, support for complex relationships between entities, and enough flexibility to evolve as the system grows.

A relational database management system (RDBMS) was selected because the problem domain is primarily composed of structured entities with well-defined relationships. Requirements engineering involves maintaining consistency between projects, functionalities, requirements, stakeholders, and documents, where modifications to one entity may affect others. Therefore, transactional operations, referential integrity, and explicit relationship modelling are important characteristics for the system.

PostgreSQL was selected due to its maturity, reliability, and extensive feature set. It provides strong relational guarantees while also supporting semi-structured data

through features such as JSONB columns. This allows storing flexible attributes when required without abandoning the advantages of a relational model.

Alternative database solutions based on document-oriented databases, such as MongoDB, were considered due to their flexible schema and ability to represent changing data structures. However, this flexibility is less relevant for the IR-Board domain, where consistency, traceability, and explicit relationships between entities are critical. A document-oriented approach would require additional application-level logic to maintain relationships and enforce integrity constraints.

Another important consideration was the integration with the selected security architecture. The Ory ecosystem used by the project relies on relational database support, and PostgreSQL is a mature option compatible with these components, as commented here. Using the same database technology across the system reduces deployment complexity, simplifies maintenance, and avoids introducing unnecessary infrastructure differences.

The final decision was PostgreSQL because it provides the required balance between consistency, flexibility, ecosystem compatibility, and long-term maintainability.

Criteria	PostgreSQL	MongoDB	Evaluation
Data consistency	Strong ACID transactions and relational integrity	Flexible schema with weaker relationship enforcement	PostgreSQL preferred
Entity relationships	Native foreign keys and relational modelling	Relationships require additional application logic	PostgreSQL preferred
Traceability	Suitable for immutable identifiers, lifecycle states, and audit information	Possible but requires additional modelling	PostgreSQL preferred
Schema flexibility	JSONB allows semi-structured fields while maintaining relations	Highly flexible document model	Both viable
Ory ecosystem compatibility	Supported and commonly used	Not suitable for the selected deployment	PostgreSQL preferred

Table 4: Database alternatives comparison

Chapter 4. Initial Project Planning and Management

4.1 Theoretical client petition

To be able to appropiately create a project's budget aligned with the needs of the client, and ensure a complete enough context, the planning was done with the following fictional scenario in mind:

The theoretical client, NorthTech Solutions, is a company located in a nearby city to where the development team operates. The company requests the creation of a customized software platform to improve its internal processes and digital management.

The project involves collaboration between different parties, including NorthTech Solutions, the development team, the company's management department, technical staff, and the future users of the application. The requested work covers the complete software lifecycle, including requirements analysis, custom development, deployment on the internal network of the company, load testing, usability testing, and technical documentation.

The objective is to deliver a functional, maintainable, and scalable solution while applying professional software engineering practices within the scope of this TFG.

4.2 Stakeholder Identification

ID	Stakeholder	Description
ST-1	NorthTech Solutions	Fictional company requesting the software solution, defining business needs and validating the final product
ST-2	Company Management Department	Area responsible for organizational decisions, providing objectives, priorities, and project approval
ST-3	Technical Department	Employees responsible for IT systems, providing technical requirements and supporting deployment
ST-4	End Users (Software Development Teams)	Teams that will use the application daily for their development activities, providing usability feedback and validating workflows
ST-5	Development Team	TFG development group responsible for the solution, analyzing, designing, developing, testing, and documenting the system
ST-6	Client Development Team	Technical personnel from the client organization who may review the solution and support future improvements
ST-7	Future Maintenance Team	Hypothetical team responsible for future evolution, requiring documentation and maintainability information

ST-8	NorthTech Systems Administration Team	Hypothetical team responsible for infrastructure management, deployment support, and system monitoring

Domain and infrastructure providers are not included as stakeholders, since the deployment environment will rely on the theoretical client's existing infrastructure and no external provisioning or management is required within the scope of this project.

4.3 OBS and PBS

This section presents the Product Breakdown Structure (PBS) and Organization Breakdown Structure (OBS) of the IR-Board project. The PBS provides a hierarchical view of the main components and modules that compose the final product, while the OBS defines the organizational structure and the distribution of responsibilities among the roles involved in the project.

The purpose of these structures is to establish a clear understanding of the system scope, its main elements, and the relationship between the product components and the theoretical teams responsible for their development and management.

4.3.1 PBS

The Product Breakdown Structure (PBS) shown below decomposes IR-Board into its main functional components. At the highest level, the system is divided into several modules that represent the main capabilities offered by the platform.

The security and access control module contains the mechanisms required to protect the system, including authentication, Relation-Based Access Control (ReBAC), and the Zero-Trust security model. The requirements management module represents the core functionality of the platform, covering functional and non-functional requirements, their lifecycle, and traceability between related elements.

Project management provides the necessary tools to organize projects, manage their functionalities, and control approval workflows. User management handles users and permissions, while the project element management module groups additional entities managed within projects, such as documents and stakeholders.

Finally, collaboration services provide support for concurrent work through entity locking, and the search and filtering module improves accessibility by allowing users to efficiently locate and organize project information.

This decomposition defines the main product boundaries and provides a high-level overview of the system architecture from a functional perspective.

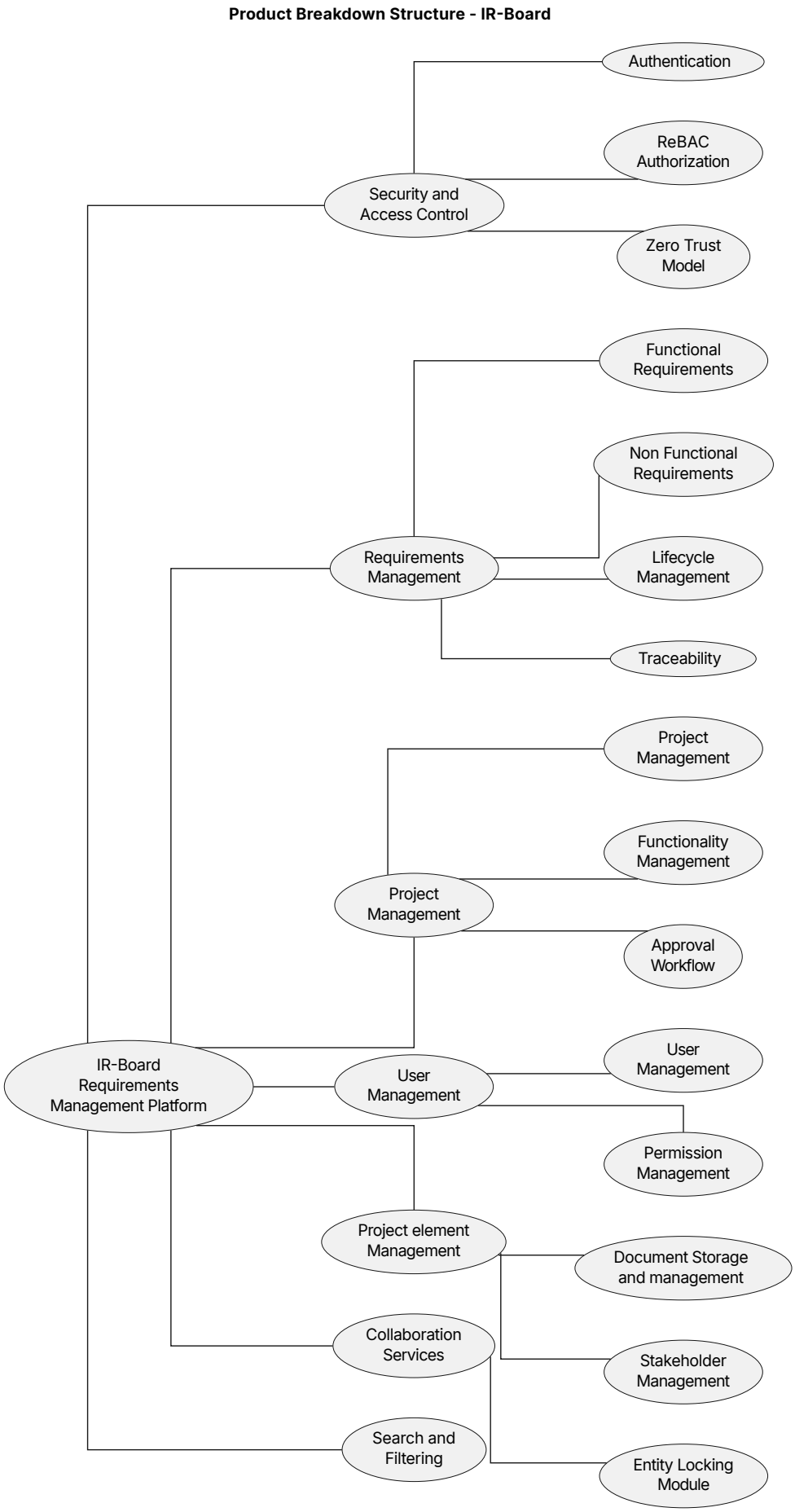
Product Breakdown Structure - IR-Board
IR-Board Requirements Management Platform
Security and Access Control
Authentication
ReBAC Authorization
Zero Trust Model
Requirements Management
Functional Requirements
Non Functional Requirements
Lifecycle Management
Traceability
Project Management
Project Management
Functionality Management
Approval Workflow
User Management
User Management
Permission Management
Project element Management
Document Storage and management
Collaboration Services
Stakeholder Management
Search and Filtering
Entity Locking Module

Figure 2: PBS

4.3.2 OBS

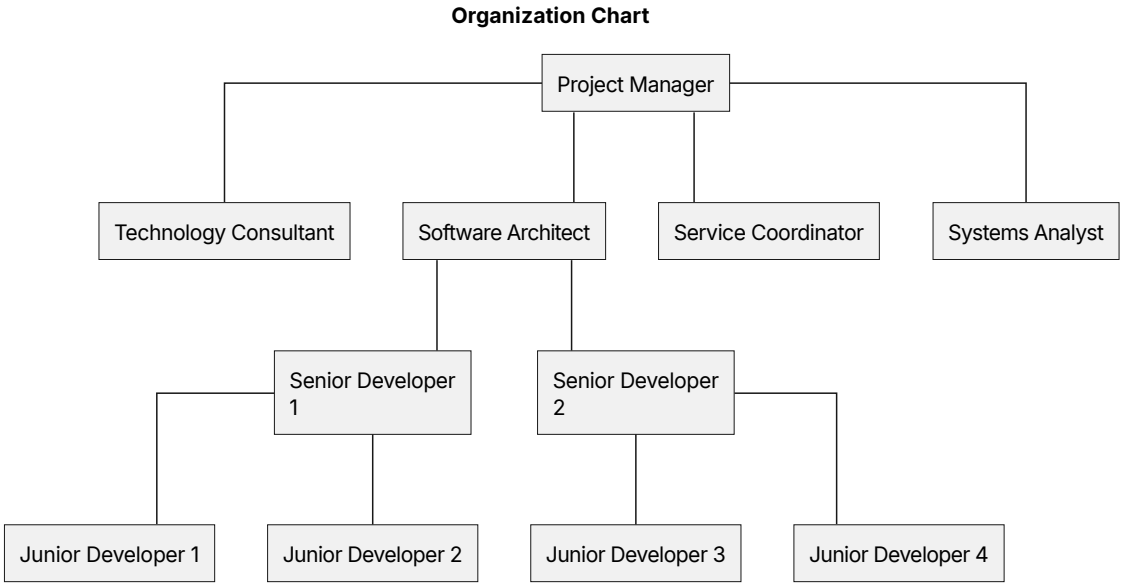

Figure 3: OBS

A RACI responsibility matrix has been used to define the responsibilities and participation of each stakeholder throughout the project. This matrix helps clarify the involvement of each role during the different project activities.

The RACI model defines the following categories:

- R (Responsible): Person or role responsible for carrying out the activity.
- A (Accountable): Person or role who approves the result and has the final responsibility.
- C (Consulted): Person or role whose expertise or opinion is required.
- I (Informed): Person or role who must be notified about the progress or results.

The roles considered in the project are:

- Project Manager (PM)
- Systems Analyst (SA)
- Service Coordinator (SC)
- Technology Consultant (TC)
- Software Architect (SWA)
- Senior Developers (SD)
- Junior Developers (JD)

The following matrix defines the responsibility distribution for the main project activities:

Activity	PM	SA	SC	TC	SWA	SD	JD
Project Management	R	I	I	I	I	I	I
Requirements Analysis	A	R	R	I	I	I	I
Software Design	I	A	C	R	R	I	I
Software Development	I	I	I	C	A	R	C

Testing and Validation	I	A	C	I	I	R	R
Documentation	A	I	I	I	C	R	R

4.4 Initial Planning

4.5 Risk Analysis

4.6 Initial Budget

Chapter 5. System Analysis

5.1 Users and Characteristics

In this project, instead of conventional system-wide roles, the approach I found fit best the nature of requirement management systems was a relation-based role system. Therefore, two sets of user permissions can be defined: system-level permissions and project-level permissions.

User	Description
Admin	Has access to project creation, deactivation, purge of removed projects... Still needs a project role to add to or modify a project, even if created by himself. Can link users as project manager to a project.
Basic	A basic user, the default. Has access to his profile management.

Table 5: List of permission levels on the system

A user is either an admin or not. Due to zero-trust, unless he is added as project manager to it he wont be able to modify any project, even if it was created by himself.

User	Description
ProjectManager	Access to add, edit and disable functionalities, requirements within those functionalities, documents, and can link users to that project.
RequirementEngineer	Linked to a functionality, has access to add, modify and disable requirements in that functionality, as well as with documents linked to the project.
Stakeholder	Linked to a functionality, has read-only access to the requirements of that functionality, and documents linked to the project.

Table 6: List of permissions within a project

These permissions overlap, in case a user is linked to a project in several ways. In that situation, due to this overlap, the most high level permission prevails.

For example, if a user is listed as requirement engineer and stakeholder in the same functionality, the user will be able to view (both permissions), and add, modify, disable and all other requirement engineer actions as well.

5.2 Requirements Analysis

Here are presented some diagrams that helped during the requirement edduction process.

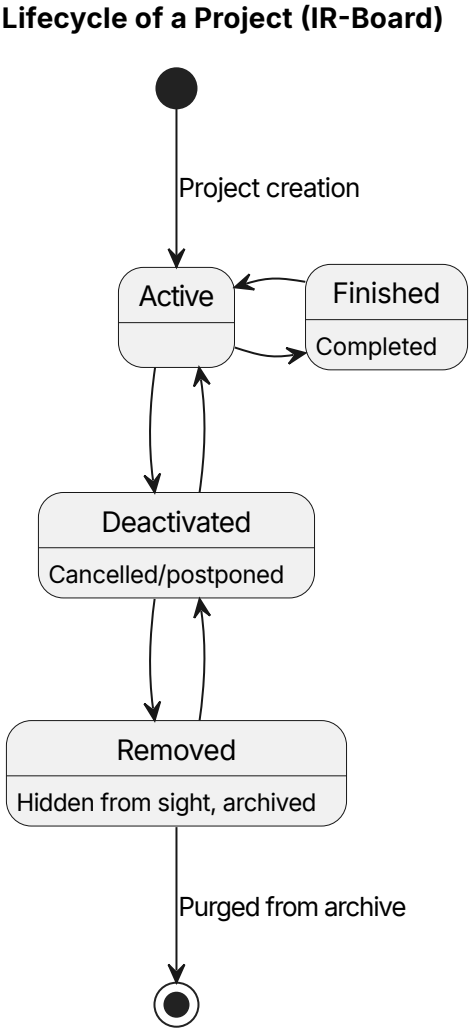

Figure 4: Lifecicle of a project entity

Active - A project entity that is currently in progress.

finished - A project entity that has been implemented. As the nature of a project is that one never ends, it does not represent an end state.

Deactivated - A project entity that has been cancelled, postponed etc; A project that is not finished but is not currently in use.

Removed - A project entity that has been archived for removal, placed in the trash bin in case it is needed as last resort.

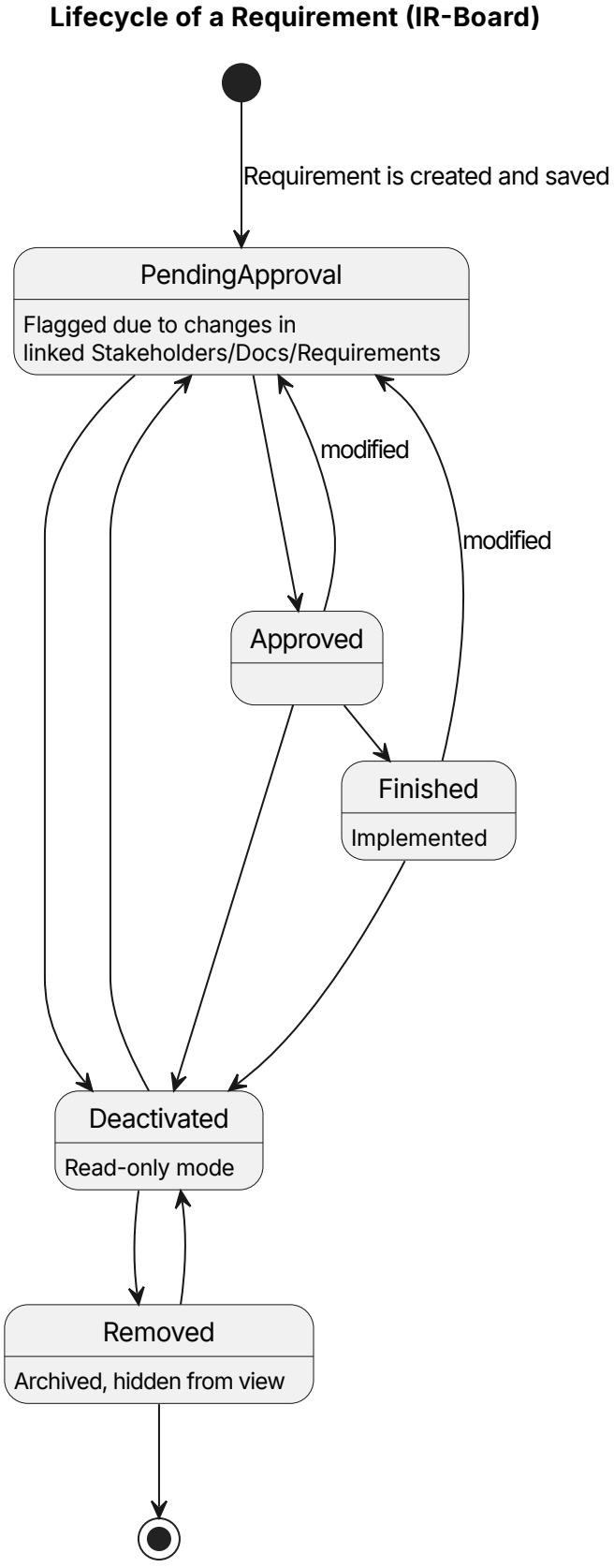

Figure 5: Requirement entity's state diagram

PendingApproval - A requirement entity that has not been validated by a stakeholder as it currently is. It's a complex state to ease development, as the pending

review can be seen as an extension of itself, and therefore can be a simple boolean flag.

PendingReview - A requirement entity that needs attention and possible modification, due to a change on a linked entity. Expected to be purely a flag.

Approved - A requirement entity that has been validated by the appropiate stakeholders with the project manager outside the system.

Deactivated - A requirement entity has been deactivated for some reason, be cancelled or an error, and does not count towards the metrics of the project.

Removed - A requirement entity that has been deemed innecessary to the project. It is hidden from view, archived.

5.3 System Analysis

5.3.1 Class Analysis

5.3.2 Data Modeling

On this document, several different kinds of identifiers are referenced, be dynamic, internal unique identifier, or “entity slug”. TODO explain more

5.3.3 Process Modeling

A crucial process for the system is the updates triggered by modifications between observed and observer entities. To illustrate the flows depicted on the system, the following diagram is provided:

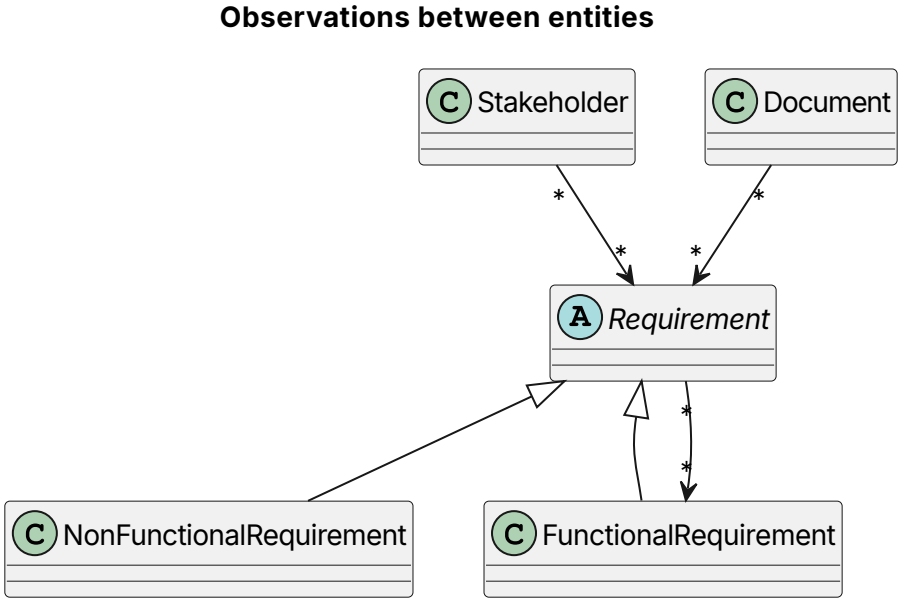

Figure 6: Possible observation processes between entities

Linking Arrows - The arrows in this diagram refer to linked entities in a modified observer pattern, to allow to search for linked elements from both sides of the relation. The direction of the arrow expresses the flow from a observed element to the observer element, for example, a modification on a Stakeholder element would trigger an update() call to all requirements observing it.

5.3.4 User Interface Definition

5.4 Requirements Specification

5.4.1 External Interfaces

5.4.2 Functional Requirements

5.4.2.1 project management

PM.1. The system must allow an admin to create a project.
- PM.1.1. The system must require a Project name to generate a project correctly.
- PM.1.2. The system must require a description to generate a project correctly.
- PM.1.3. The system must require a user to be an admin of the system to generate a project correctly.
- PM.1.4. The system must allow to optionally change the priority style set for the project, either:
  - PM.1.4.1. Ternary (High, Medium, Low) Predefined.
  - PM.1.4.2. MOSCOW (Must, Should, Could or Won't have).

PM.2. The system must allow an admin to deactivate a project.
- PM.2.1. The system must ask for confirmation before deactivating.
- PM.2.2. The system must put the project on read only mode.

PM.3. The system must allow an admin to reactivate a deactivated project.

PM.4. The system must allow an admin to modify an active project.

PM.5. The system must allow to link users with a project.
- PM.5.1. The system must allow an admin to link users to a project as project manager.
- PM.5.2. The system must allow an admin or project manager linked to the project to link users to a functionality on said project as a stakeholder user.
- PM.5.3. The system must allow a project manager to link users to one or more functionalities of a project as requirement engineers.

PM.6. The system must allow access to the project description/dashboard to users linked to it or a functionality of it.
- PM.6.1. The system must show the total split of requirements by their states (pie chart).
- PM.6.2. The system must not take into account deactivated requirements toward any metric.
- PM.6.3. The system must show the different functionalities of the project.

PM.7. The system must allow a project manager to mark as approved all elements in a project.

PM.8. The system must allow a project manager to add a functionality to a project.
- PM.8.1. A functionality needs a name and unique set of letters for its dynamic identifier.
- PM.8.2. The system must automatically attempt to get the letters for the dynamic identifier from the name.

PM.8.2.1. The system must take the first letter from every word in the name.

PM.8.3. The system must deny adding any functionality with an identifier matching another on the same project.

PM.8.4. The system must allow to use a custom functionality identifier.

PM.8.5. The system must automatically link the project manager to the new functionality.

PM.9. The system must allow a project manager to modify a functionality.

PM.10. The system must allow a project manager to deactivate a functionality.

PM.10.1. The system must ask for confirmation before deactivating.

PM.10.2. The system must put the functionality (elements contained by it) on read only.

PM.11. The system must allow a project manager to reactivate a functionality.

PM.12. The system must allow a project manager to mark as approved all elements in a functionality.

PM.13. The system must allow a project manager to export the project's requirements onto a pdf file.

5.4.2.2 Stakeholders management

SM.1. The system must allow any user linked with the project access to view its stakeholders.

SM.1.1. The system must show stakeholders that are not removed.

SM.1.2. The system must show if a stakeholder is flagged as pending review.

SM.1.3. The system must show the identifier, the name and part of the description.

SM.1.4. The system must allow to view the details of a stakeholder.

SM.1.4.1. The system must show all atributes of a stakeholder.

SM.1.4.2. The system must show all requirements linked to it.

SM.2. The system must allow a project manager to view its removed stakeholders.

SM.3. The system must allow a requirement engineer or a project manager to add a new stakeholder to a project.

SM.3.1. The system must only allow a stakeholder to be added to a project the user is linked to.

SM.3.2. The system must require a name to generate a stakeholder.

SM.3.3. The system must require a description to generate a stakeholder.

SM.3.4. The system must generate the identifier for the stakeholder.

SM.4. The system must allow a project manager or requirement engineer to link a stakeholder to one or more requirements on the same project.

SM.4.1. The system must only allow the user to link the stakeholder to a requirement on a functionality they are linked to.

SM.5. The system must allow a project manager or requirement engineer to unlink a stakeholder from one or more requirements.

SM.5.1. The system must only allow the user to unlink a stakeholder from a requirement of a functionality they are linked to.

SM.6. The system must allow a requirement engineer or a project manager to deactivate a stakeholder from a project the user is linked to.

SM.6.1. The system must flag all entities linked as pending review.
SM.6.2. The system must put the stakeholder on read only mode.
SM.7. The system must allow a requirement engineer or a project manager to enable a disabled stakeholder from a project the user is linked to.
SM.7.1. The system must flag all entities linked as pending review.
SM.7.2. The system must put the stakeholder on read only mode.
SM.8. The system must allow a project manager to remove a disabled stakeholder from a project the user is linked to.
SM.8.1. The system must flag all entities linked as pending review.
SM.8.2. The system must unlink any requirements linked to the stakeholder
SM.9. The system must allow a project manager to delete permanently a removed stakeholder.
SM.10. The system must allow a project manager or requirement engineer to modify a stakeholder that's not deactivated or removed.
SM.10.1. The system must flag as pending review linked entities upon saving.

5.4.2.3 Requirement management

RM.1. The system must allow users linked to a project or functionality of a project access to its requirements.
RM.1.1. The system must only allow users linked to the functionality of a functional requirement access to it.
RM.1.2. The system must allow to collapse and expand requirements with children.
RM.1.3. The system must show the dynamic identifier, the name, state and part of the description.
RM.1.4. The system must allow to view the details of a requirement.
RM.1.4.1. The system must show the internal unique identifier.
RM.1.4.2. The system must show all atributes of a requirement.
RM.1.4.3. The system must show all stakeholders linked to it.
RM.1.4.4. The system must show all requirements cross-linked with it.
RM.1.4.5. The system must show all documents linked to it.
RM.2. The system must allow a requirement engineer or a project manager to add a requirement to a project the user is linked to.
RM.2.1. The system must only allow a functional requirement to be added to a functionality the user is linked to.
RM.2.2. The system must allow the user to generate a requirement as a child of another requirement (nesting).
RM.2.3. The system must assign automatically the dynamic identifier.
RM.2.3.1. The identifier must be based on its relation to other requirements.
RM.2.3.2. The identifier must represent if it is a functional or non functional requirement (FR or NFR).
RM.2.3.3. The identifier must represent the folder/component that holds the requirement (user management → UM).
RM.2.4. The system must assign automatically an internal unique slug.

RM.2.4.1. The identifier must represent the project that will hold the requirement.
RM.2.4.2. The identifier must represent whether the requirement is functional or non functional.
RM.2.4.3. The identifier must have a random element to ensure a low colision rate.
RM.2.5. The system must ask for the following data for a functional requirement:
RM.2.5.1. The system must require a name.
RM.2.5.2. The system must require a description.
RM.2.5.3. The system must require a priority following the one set on the project creation.
RM.2.5.4. The system must allow to set a stability, which is optional.
RM.2.5.5. The system must allow to set a stakeholder as origin, which is optional.
RM.2.6. The system must ask for the following data for a non-functional requirement:
RM.2.6.1. The system must require a name.
RM.2.6.2. The system must require a description.
RM.2.6.3. The system must allow to set a measurement unit.
RM.2.6.4. The system must allow to set a comparison operator.
RM.2.6.4.1. equal to, less than or greater than.
RM.2.6.5. The system must allow to set a threshold value.
RM.2.6.5.1. This value represents the minimum value to mark the requirement as passed.
RM.2.6.6. The system must allow to set a target value.
RM.2.6.6.1. This value represents the optimal value desired by the team.
RM.2.6.7. The system must allow to set an actual value.
RM.2.6.7.1. This value represents the current status of the measurement.
RM.2.7. The system must automatically set the new requirement as pending approval.
RM.3. The system must allow a project manager or requirement engineer to link a requirement on a functionality they are linked to, to another entity.
RM.3.1. The system must allow to link a requirement with a stakeholder of the same project.
RM.3.2. The system must allow to un-link a requirement with a stakeholder.
RM.3.3. The system must allow to link a requirement with one or more requirements of functionalities of the same project the user is linked to.
RM.3.4. The system must allow to un-link a requirement with other requirements of functionalities of the same project the user is linked to.
RM.3.5. The system must allow to link a requirement with one or more documents of the same project.

RM.3.6. The system must allow to un-link a requirement with one or more documents of the same project.

RM.4. The system must allow a requirement engineer or a project manager to deactivate a requirement pending approval on a functionality they are linked to.

RM.4.1. The system must flag any requirements linked to the deactivated requirement as pending review.

RM.4.2. The system must put the requirement on read only.

RM.5. The system must allow a project manager or requirement engineer to set a deactivated requirement as removed.

RM.5.1. The system must hide from view a removed requirement, effectively archiving it.

RM.6. The system must allow a project manager to permanently delete a removed requirement.

RM.6.1. The system must hide from view a removed requirement, effectively archiving it.

RM.7. The system must allow a requirement engineer or a project manager to reactivate a requirement on a functionality they are linked to.

RM.7.1. The system must automatically flag as pending review the reactivated requirement.

RM.8. The system must allow a requirement engineer or a project manager to modify a requirement on a project.

RM.8.1. The system must only allow a project manager or requirement engineer to modify functional requiremets of a functionality the user is linked to.

RM.8.2. The system must flag linked requirements as pending review upon saving with changes.

RM.9. The system must allow a project manager to mark as approved one or more requirements.

RM.9.1. The system must only allow to mark as approved a requirement that is pending approval, not pending review nor deactivated.

RM.10. The system must allow a project manager to mark as finished a requirement.

RM.10.1. The system must set the requirement as pending approval if it is modified.

RM.11. The system must allow a project manager or requirement engineer linked to a functionality of the project to change the position of a requirement.

RM.11.1. The system must allow reordering of functional requirements to users linked to the same functionality.

RM.11.2. The system must update the dynamic identifier automatically.

RM.11.3. The system must set the order of requirements using a floating point order value.

5.4.2.4 User management

UM.1. The system must allow an admin to invite new users to the system.

UM.1.1. The system must provide different levels of authorisation, based on the relationship between its elements.

UM.1.1.1. The system must have the levels: Admin, project manager, requirement engineer and stakeholder user.

UM.1.2. The system must ask the admin to set the name, surname, and email of the invited user.

UM.1.2.1. The system must generate an signup code as a temporal password.

UM.1.2.2. The system must automatically send an invitation with the signup code to the email of the invited user.

UM.2. The system must allow an admin to view the name and surname of a user from the system.

UM.3. The system must allow an admin to modify the name and surname of a user from the system.

UM.4. The system must allow an admin to view the current permissions of a user from the system.

UM.5. The system must allow an admin to generate a new invite with a signup code for a user.

UM.6. The system must allow any user with valid credentials to sign in to the system.

UM.6.1. The system must prompt any user signing in with a signup code to set a permanent password.

UM.6.1.1. The system must ensure the password is between 15 and 64 characters long.

UM.6.1.2. The system must make use of a random salt specific of each user.

UM.6.1.3. The system must remove any password or signup code of the user upon seting a permanent password.

UM.6.2. The system must temporally block the user after 3 consecutive failed attempts.

UM.7. The system must allow an admin to remove a user from the system.

UM.7.1. The system must clean the removed user's ReBAC permissions.

5.4.2.5 Document management and modelling

DMM.1. The system must allow users linked to a project access to documents of that project.

DMM.1.1. The system must show documents not removed.

DMM.1.2. The system must show entities linked to the document.

DMM.2. The system must allow a project manager to access removed documents of that project.

DMM.2.1. The system must show entities linked to the document.

DMM.3. The system must allow a project manager or a requirement engineer to add document to a project.

DMM.3.1. The user must be linked to the project.

DMM.4. The system must allow a document to be linked to one or more requirements of the same project.

DMM.4.1. The system must flag those requirements linked to it as pending a review if the document is altered.

DMM.5. The system must allow a project manager or requirement engineer to update a document.
- DMM.5.1. The user must be linked to the project the document is on.
- DMM.5.2. The system must flag as pending a review any requirements linked to the document.

DMM.6. The system must allow a project manager or requirement engineer to disable a document.
- DMM.6.1. The system must flag as pending a review any requirements linked to the document.

DMM.7. The system must allow a project manager or requirement engineer to enable a disabled document.
- DMM.7.1. The system must flag as pending a review any requirements linked to the document.

DMM.8. The system must allow a project manager or requirement engineer to remove a disabled document.
- DMM.8.1. The system must flag as pending a review any requirements linked to the document.
- DMM.8.2. The system must unlink any requirements linked to the document.

DMM.9. The system must allow a project manager to delete permanently a removed document.

5.4.2.6 Concurrency

C.1. The system must block other users from modifying an entity that another user is already modifying.
- C.1.1. The system must release automatically the entity if the user modifying it saves and exits (stops modifying).
- C.1.2. The system must release automatically the entity after a predetermined timeout period.
- C.1.3. The system must release automatically the entity if the user editing it modifies another entity.
- C.1.4. The system must only accept changes to the entity from the user who holds the entity.

C.2. The system must display for other users who is modifying the entity.

5.4.2.7 Search and filtering

SF.1. The system must allow searching a project entity by its internal unique identifier.
- SF.1.1. The system must search lexically.
- SF.1.2. The system must allow the user to see the details of the found entity.
  - SF.1.2.1. only if an exact match occurs,
  - SF.1.2.2. only if the user has access to it.

SF.2. The system must allow users to filter entities they have access to.
- SF.2.1. The system must allow filtering out deactivated requirements.
- SF.2.2. The system must allow filtering requirements based on priority.
- SF.2.3. The system must allow filtering requirements based on state.
- SF.2.4. Any filter must be reversible; ascending or descending order.

5.4.3 Usability Requirements

5.4.4 Performance Requirements

5.4.5 Logical Database Requirements

5.4.6 Design Constraints

5.4.7 System Attributes

5.4.8 Supporting Information

5.5 Test Plan Analysis

To ensure the reliability, maintainability, and performance of the IR-Board system, a multi-dimensional testing strategy has been defined. This plan covers the entire development lifecycle, from code quality to system behavior under stress.

5.5.1 Code maintainability and unit testing with SonarQube

The project utilizes SonarQube as a Static Application Security Testing (SAST) tool. This analysis is integrated into the development workflow to ensure the following:

- Maintainability: Identification of "code smells" and technical debt that could hinder future scalability.
- Code Coverage Validation: Monitoring the percentage of the source code executed during automated tests. This ensures that critical business logic is thoroughly verified, maintaining a high safety net against regressions and establishing a minimum threshold of tested code before deployment.
- Reliability: Detection of potential bugs and logic errors through automated pattern matching.
- Security: Scanning for common vulnerabilities and ensuring compliance with industry standards (e.g., OWASP Top 10).

5.5.2 Load testing

For the load testing phase, the primary focus will be on stressing the critical entry points of the system, specifically the traffic flow passing through Traefik and Oathkeeper toward the Spring Boot backend. The goal is to simulate bursts of concurrent users to identify the exact point where identity validation latency begins to degrade the user experience or if Kratos' session management can handle the expected volume. This process goes beyond checking for server crashes; it involves using the Grafana stack to monitor how container resources scale and ensuring the internal network routing maintains stability under heavy pressure.

5.5.3 Usability testing

Regarding usability testing, the plan involves observing real users interacting with the React interface to validate that the integration of Grafana dashboards feels intuitive and seamless. Special attention will be paid to how easily users can navigate between Loki logs and the core business logic, ensuring that the underlying complexity of the microservices architecture remains completely transparent to the end user. The ultimate objective is to confirm that authentication flows do not create unnecessary friction and that the frontend information hierarchy allows for efficient data management without requiring prior technical knowledge from the operator.

Chapter 6. System Design

6.1 System Architecture

6.1.1 General architecture desing

The system architecture follows a Microservices approach based on the Zero Trust security model. This ensures flexibility and scalability while maintaining a high level of isolation between business logic and infrastructure concerns. To guarantee a professional security standard while maintaining a manageable project scope, core identity and access management responsibilities have been delegated to the Ory Open Source ecosystem.

The figure below shows the main flow of the application represented by solid arrows, and secondary messaging between microservices represented by a dotted arrows.

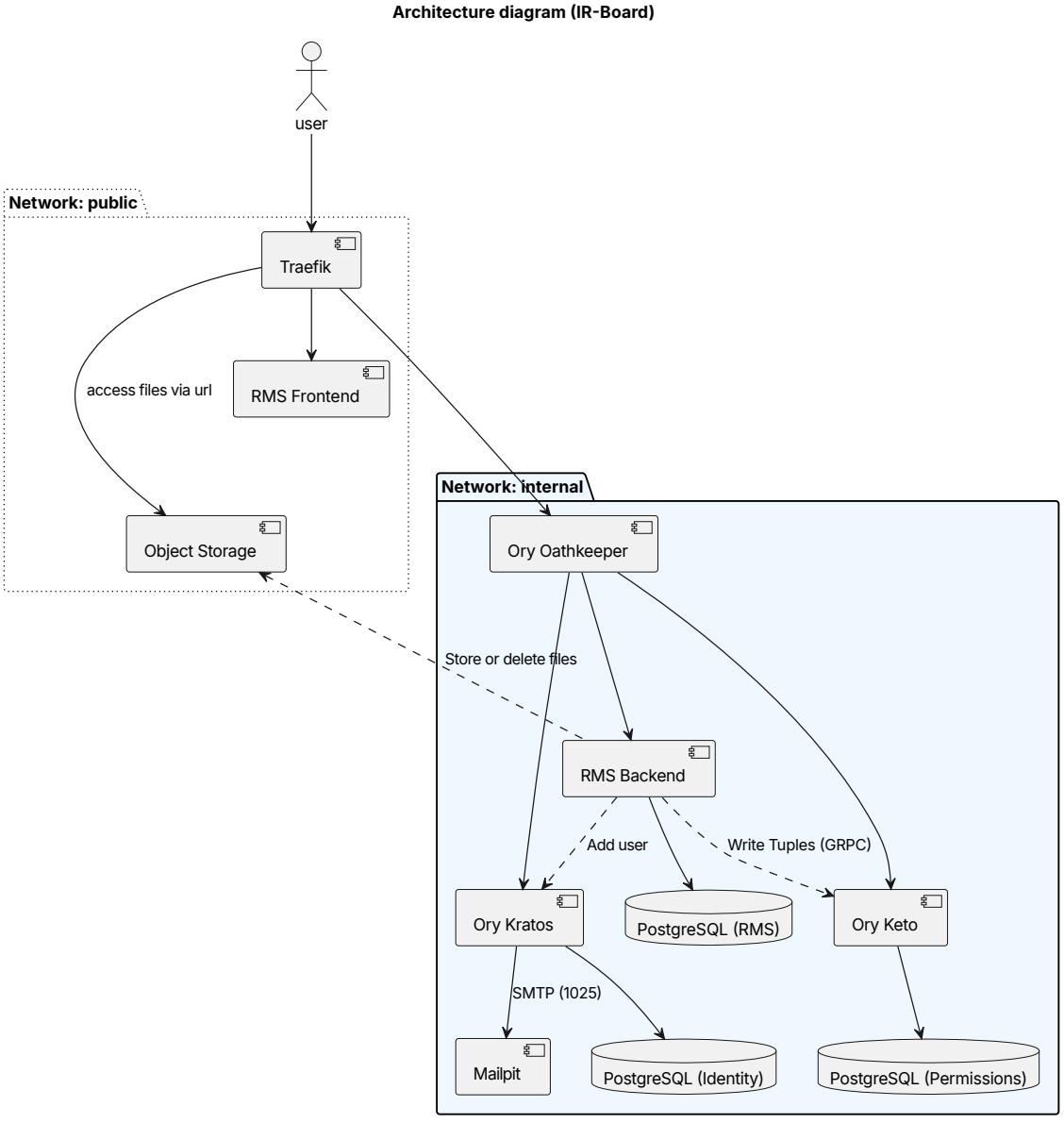

Figure 7: Architecture C2 component diagram

Traefik - Acts as the system's entry point and TLS Termination Proxy. It handles dynamic routing and load balancing, effectively hiding the internal network topology and eliminating the need to expose multiple ports to the public internet.

RMS Frontend - Built with React and TypeScript, served as static content. It executes within the user's browser and communicates with the backend services through the API Gateway.

Ory Oathkeeper - A policy-enforcement engine that acts as a gatekeeper between the public and internal networks. It intercepted every request to validate session integrity (via Kratos) and fine-grained permissions (via Keto) before allowing traffic to reach the internal services.

Ory Keto - A relationship-based access control (ReBAC) server inspired by Google's Zanzibar. It manages permission tuples, allowing the system to verify complex authorization rules (e.g., checking if a user is linked to a specific project).

Ory Kratos - Manages the full identity lifecycle, including user registration, multi-factor authentication, and session management, ensuring that sensitive credentials are handled by a specialized security component.

RMS Backend - The core service developed using Spring Boot, containing the domain-specific business logic and data persistence. It interacts with keto both to write Relation-Based Access Control (ReBAC) tuples and to filter by permissions.

Mailpit - A simple email server that receives all messages sent by Ory Kratos. Acts as a placeholder for development instead of a real email server, to ensure the signup works. TODO add the other containers TODO add another C1, C3

6.1.2 Backend system design

6.1.3 Frontend system design

6.2 Real Use Case Design

6.3 Class Design

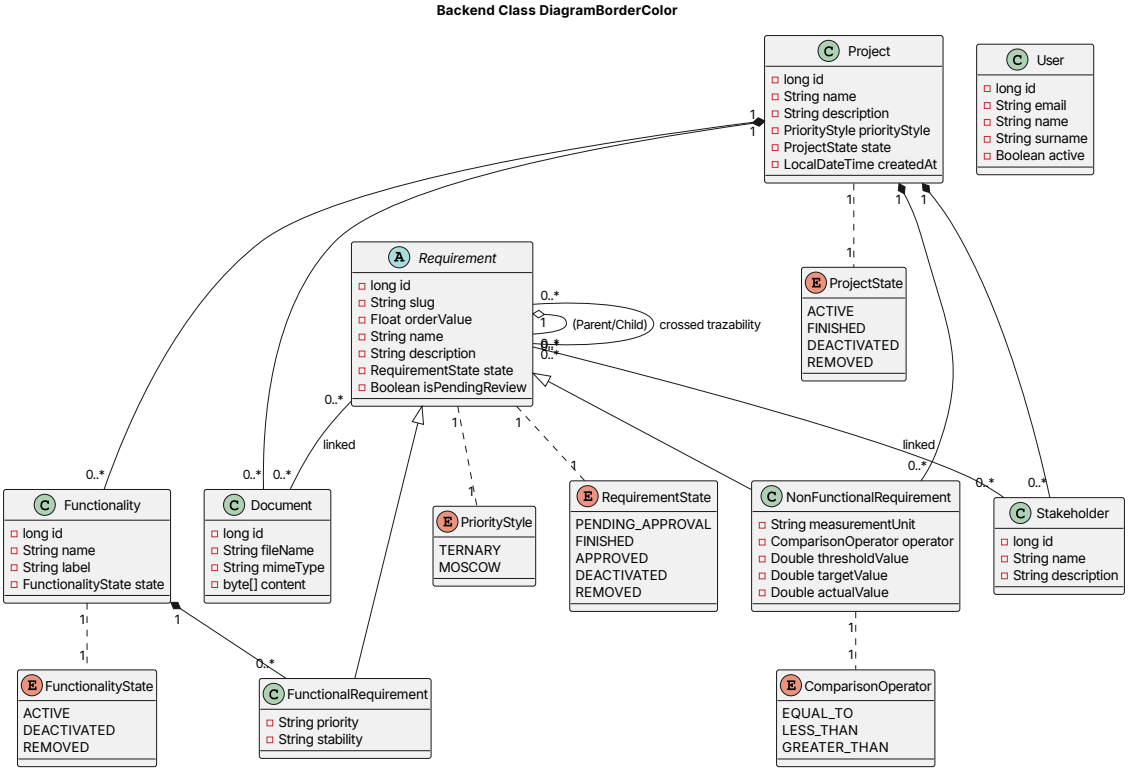

Figure 8: Domain class diagram

User - The relationships between User and Project and Functionality, as they are purely access control related, are delgated to ory Keto or whatever security ReBAC system used. The boolean value isActive is also delegated to the ReBAC system, as it represents a user-to-system relationship.

6.4 Database Design

(explain both jpa resulting database and keto relationship db)

6.5 User Interface Design

To design the user interfaces, the design tool moqups was used to model the UI wireframes. All designs provided were shared and accepted by the tutors, and are free to be inspected here:

TFG moqups

6.6 Test Plan Specification

Chapter 7. System Implementation

Chapter 8. Test Plan Execution

8.1 Unit Testing

8.2 Integration and System Testing

8.3 Usability Testing

8.4 Accessibility Testing

8.5 Load Testing

8.6 Acceptance Testing

Chapter 9. System manuals

9.1 Installation Guide

9.2 User Manual

9.3 Developer Guide

Chapter 10. Project Closure

10.1 Final Schedule

10.2 Final Risk Report

10.3 Final Budget

10.4 Project Closure Analysis

Chapter 11. Conclusions and Future Work

Chapter 12. References

- [1] “Welcome to Ory! | Ory,” Ory.com, Oct. 15, 2025. https://www.ory.com/docs/ (accessed Mar. 07, 2026).
- [2] “Configuring Vite,” vitejs, 2025. https://vite.dev/config/
- [3] “Typst Documentation,” Typst, 2024. https://typst.app/docs/

Chapter 13. Appendices

13.1 Supplementary Material